

Question:	1	2	3	Total
Points:	10	10	10	30
Score:				

Instructor: Dr. Ali Alsaffar

(1) Answer all of the following questions.

- (a) [3 points] Consider the time complexity $T(n) = 3T\left(\frac{n}{2}\right) + 4n$. Find the order of growth of $T(n)$ using the Master Theorem.

Solution: $a = 3$, $b = 2$, $d = 1$ and $b^d = 2^1 = 2$; since $a > b^d \Rightarrow T(n) \in \Theta(n^{\log_2 3})$.

- (b) [3 points] Write the time complexity recurrence $T(n)$ of *Mergesort* for both the best and worst cases, where n is the input size.

Solution: Best Case: $T(n) = 2T(n/2) + \lfloor n/2 \rfloor$
Worst Case: $T(n) = 2T(n/2) + n - 1$

- (c) [2 points] Give an example of two subarrays that cause the MERGE($A[0..n-1]$) algorithm to reach its worst case.

Solution: Left Subarray: [1, 3, 5, 7]
Right Subarray: [2, 4, 6, 8]

- (d) [2 points] How many multiplications are required to compute x^{39} using divide-and-conquer? Show how?

Solution: The solution is

$x^{39} = x * x^{19} * x^{19}$ (two multiplications)
 $x^{19} = x * x^9 * x^9$ (two multiplications)
 $x^9 = x * x^4 * x^4$ (two multiplications)
 $x^4 = x^2 * x^2$ (one multiplication)
 $x^2 = x * x$ (one multiplication)
 Total Multiplications = 8.

$Pow(x, 39) = x * Pow(x * x, 19)$ (two multiplications)
 $Pow(x * x, 19) = x * Pow(x * x, 9)$ (two multiplications)
 $Pow(x, 9) = x * Pow(x * x, 4)$ (two multiplications)
 $Pow(x * x, 4) = Pow(x * x, 2)$ (one multiplication)
 $Pow(x, 2) = Pow(x * x, 1)$ (one multiplication)
 Total Multiplications = 8.

- (2) (a) [5 points] Draw a diagram showing how the Mergesort algorithm (Divide-and-Conquer) will sort the array $A : 21, 23, 22, 20, 24, 26, 25, 27$.

Solution:

- (b) [5 points] Re-write the partition algorithm PARTITION($A[l..r]$) of Quicksort using **while** loops only.

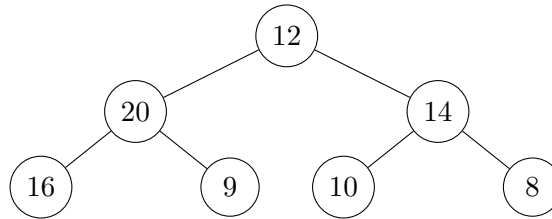
Solution:

```

1: Algorithm PARTITION( $A[l..r]$ )
2:    $p \leftarrow A[l], i \leftarrow l, j \leftarrow r + 1$ 
3:   while  $i < j$  do
4:      $i \leftarrow i + 1$ 
5:     while  $A[i] < p$  and  $i \leq r$  do  $i \leftarrow i + 1$ 
6:      $j \leftarrow j - 1$ 
7:     while  $A[j] > p$  do  $j \leftarrow j - 1$ 
8:     if  $i \leq j$  then
9:        $\text{SWAP}(A[i], A[j])$ 
10:     $\text{SWAP}(A[l], A[j])$ 
11:  return  $j$ 

```

(3) Consider the following binary tree:



(a) [2 points] Explain why this tree is *not* a max-heap?

Solution: The tree violates the max-heap property because the root has key 12, but its left child has key 20 which is larger. In a max-heap, each parent key must be greater than or equal to those of its children.

(b) [3 points] Write the array representation $H[1..7]$ of the tree.

	1	2	3	4	5	6	7
$H \rightarrow$	12	20	14	16	9	10	8

(c) [5 points] Apply one **HEAPIFY** operation starting at the root. After each swap, draw the updated tree, then write the array H once the process is complete.

Solution: Apply **MAX-HEAPIFY**($H, 1$):

1. Compare $H[1] = 16$ with its children $H[2] = 20$ and $H[3] = 14$. Largest value is $H[2] = 20$. Swap $H[1] \leftrightarrow H[2]$.

$$H = [20, 16, 14, 12, 9, 10, 8].$$

2. Now call **MAX-HEAPIFY**($H, 2$) on the affected subtree. $H[2] = 16$ has children $H[4] = 12$ and $H[5] = 9$. Largest is 16 (already parent), so stop.

Final heap after a single **HEAPIFY**:

$$H = [20, 16, 14, 12, 9, 10, 8].$$

The tree now satisfies the **max-heap property**.